

PDF-Based Q&A with LangChain and FAISS

This notebook demonstrates how to load a PDF file, split its content into semantic chunks, generate embeddings using HuggingFace, store them in a FAISS vector store, and perform semantic search for intelligent retrieval.

```
[1]
✓ 10s
!pip install -q \
langchain==0.2.16 \
langchain-core==0.2.38 \
langchain-community==0.2.16 \
langchain-text-splitters==0.2.4 \
sentence-transformers \
faiss-cpu \
pypdf
```

```
[2]
✓ 48s
!pip install numpy==1.26.4
!pip install sentence-transformers==2.7.0
!pip install transformers==4.41.2
```

```
... Requirement already satisfied: numpy==1.26.4 in /usr/local/lib/python3.12/dist-packages (1.26.4)
Collecting sentence-transformers==2.7.0
  Downloading sentence_transformers-2.7.0-py3-none-any.whl.metadata (11 kB)
Collecting transformers<5.0.0,>=4.34.0 (from sentence-transformers==2.7.0)
  Downloading transformers-4.57.6-py3-none-any.whl.metadata (43 kB)
    44.0/44.0 kB 2.3 MB/s eta 0:00:00
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from sentence-transformers==2.7.0) (4.67.3)
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers==2.7.0) (2.11.0)
```

[3] ✓ 1s

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA
```

[4] ✓ 0s

```
loader = PyPDFLoader("/content/Deep_learning_notes.pdf")
```

[5] ✓ 7s

```
docs = loader.load()
```

[6] ✓ 0s

```
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=300)
```

[7] ✓ 0s

```
chunks = splitter.split_documents(docs)
```

[8] ✓ 10s

```
pip install sentence-transformers
```

... Requirement already satisfied: sentence-transformers in /usr/local/lib/python3.12/dist-packages (2.7.0)
Requirement already satisfied: transformers<5.0.0,>=4.34.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.67.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.67.3)
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (2.11.0+cpu)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.26.4)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.6.1)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.16.3)
Requirement already satisfied: huggingface-hub>=0.15.1 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (0.3)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (11.3.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.15.1->sentence-transfor

[9]
✓ 11s

```
embeddings = HuggingFaceEmbeddings(model_name = "all-MiniLM-L6-v2")
```

/tmp/ipykernel_9234/4186238865.py:1: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain 0.2
embeddings = HuggingFaceEmbeddings(model_name = "all-MiniLM-L6-v2")

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as
You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(
modules.json: 100% ██████████ 349/349 [00:00<00:00, 6.36kB/s]

config_sentence_transformers.json: 100% ██████████ 116/116 [00:00<00:00, 4.16kB/s]

README.md: █████ 10.5k/? [00:00<00:00, 150kB/s]

sentence_bert_config.json: 100% ██████████ 53.0/53.0 [00:00<00:00, 636B/s]

/usr/local/lib/python3.12/dist-packages/huggingface_hub/file_download.py:949: FutureWarning: `resume\_download` is deprecated and will be removed in version 0.11.0. To get the same behavior, please use `resume\_download=False`.
warnings.warn(
config.json: 100% ██████████ 612/612 [00:00<00:00, 29.2kB/s]

model.safetensors: 100% ██████████ 90.9M/90.9M [00:02<00:00, 51.9MB/s]

tokenizer_config.json: 100% ██████████ 350/350 [00:00<00:00, 22.1kB/s]

vocab.txt: █████ 232k/? [00:00<00:00, 6.07MB/s]

tokenizer.json: █████ 466k/? [00:00<00:00, 12.9MB/s]



```
[10] ✓ 1m vectors = FAISS.from_documents(chunks, embeddings)
```

```
[11] ✓ 0s retriever = vectors.as_retriever(search_kwargs={"k": 3})
```

```
[12] ✓ 0s query = "What is the main idea of this document?"  
results = retriever.get_relevant_documents(query)
```

```
✓ /tmp/ipykernel_9234/412995169.py:2: LangChainDeprecationWarning: The method `BaseRetriever.get_relevant_documents` was deprecated i  
results = retriever.get_relevant_documents(query)
```

```
[13] ✓ 0s print(f"\n🔥 Top results for: {query}")  
for i, doc in enumerate(results, 1):  
    print(f"\n--- Chunk {i} ---\n{doc.page_content}")
```

```
✓ ... 🔥 Top results for: What is the main idea of this document?
```

```
--- Chunk 1 ---  
the sequence element.
```

This bidirectional processing is particularly beneficial for tasks where understanding the full context of a sequence is important. For example, in named entity recognition, determining whether a word is a person's name might require information about both preceding and following words. Similarly, in speech recognition or part-of-speech tagging, knowing what comes after a word can be just as

```
--- Chunk 2 ---  
It serves as the information bottleneck which force the network to prioritize the most significant  
features. This compact representation helps the model learn the underlying structure and key patterns
```

[13]
✓ Os

```
print(f"\n🔥 Top results for: {query}")
for i, doc in enumerate(results, 1):
    print(f"\n--- Chunk {i} ---\n{doc.page_content}")
```

▼

🔥 Top results for: What is the main idea of this document?

--- Chunk 1 ---
the sequence element.

This bidirectional processing is particularly beneficial for tasks where understanding the full context of a sequence is important. For example, in named entity recognition, determining whether a word is a person's name might require information about both preceding and following words. Similarly, in speech recognition or part-of-speech tagging, knowing what comes after a word can be just as

--- Chunk 2 ---
It serves as the information bottleneck which forces the network to prioritize the most significant features. This compact representation helps the model learn the underlying structure and key patterns of the input helps in enabling better generalization and efficient data encoding.

3. Decoder
It is responsible for taking the compressed representation from the latent space and reconstructing it back into the original data form.

--- Chunk 3 ---
choosing the correct translation for a word. Similarly, in document classification or sequence tagging, future context often plays a role in understanding the meaning or function of a word. BiRNNs allow models to make more informed decisions by considering both directions simultaneously. Like BiLSTMs, BiRNNs are not suitable for tasks that require online or real-time processing since they

pdf-based-retrieval.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

RAM Disk

Files

..

sample_data

Deep_learning_notes.pdf

Disk 86.74 GB available

BiLSTMs, BiRNNs are not suitable for tasks that require online or real-time processing since they

[14] ✓ 0s query = "tell me about the biological neuron in deep learning?"
results = retriever.get_relevant_documents(query)

[15] ✓ 0s print("Answer")
print(results[0].page_content)

Answer
layers to learn patterns from large amounts of data. It is inspired by the structure and function of the human brain. Deep learning is especially powerful in handling unstructured data like images, audio, video, and text.

Deep learning models automatically learn features from raw data by passing it through multiple layers (input → hidden layers → output). Each layer learns different levels of abstraction.

Why Deep Learning?
■ Can process massive datasets

[16] ✓ 0s query = "tell me about Gradient Descent?"
results = retriever.get_relevant_documents(query)

[17] ✓ 0s print("Answer")
print(results[0].page_content)

Answer
12

pdf-based-retrieval.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

..

sample_data

Deep_learning_notes.pdf

Disk 86.74 GB available

Why Deep Learning?
Can process massive datasets

[16] ✓ 0s
query = "Tell me about Gradient Descent?"
results = retriever.get_relevant_documents(query)

[17] ✓ 0s
print("Answer")
print(results[0].page_content)

Answer
12

Stochastic Gradient Descent (SGD) is a fundamental optimization technique used in training deep learning models. Its primary goal is to minimize the loss function by adjusting the weights and biases in a neural network in such a way that the network's predictions become closer to the actual target values. Unlike Batch Gradient Descent, which computes the gradient of the loss function using

[17] ✓ 0s
Start coding or generate with AI.

Share

RAM
Disk